```
            var viewModelDate = detail.LastModified;                ❺
            context.HttpContext.Response                            ❺
              .GetTypedHeaders().LastModified = viewModelDate;      ❺
        }
    }
}
```

❶ ResultFilterAttribute provides a useful base class you can override.

❷ You could also override the Executed method, but the response would already be sent by then.

❸ Checks whether the action result returned a 200 Ok result with a view model.

❹ Checks whether the view model type is RecipeDetailViewModel . . .

❺ . . . and if it is, fetches the LastModified property and sets the Last-Modified header in the response

I've used another helper base class here, `ResultFilterAttribute`, so you need to override only a single method to implement the filter. Fetch the current `IActionResult`, exposed on `context.Result`, and check that it's an `OkObjectResult` instance with a `RecipeDetailViewModel` value. If it is, fetch the `LastModified` field from the view model and add a `Last-Modified` header to the response.

**TIP** `GetTypedHeaders()` is an extension method that provides strongly typed access to request and response headers. It takes care of parsing and formatting the values for you. You can find it in the `Microsoft.AspNetCore.Http` namespace.

As with resource and action filters, result filters can implement a method that runs *after* the result has executed: `OnResultExecuted`. You can use this method, for example, to inspect exceptions that happened during the execution of `IActionResult`.

**WARNING** Generally, you can't modify the response in the `OnResultExecuted` method, as you may have already started